



Islamic University of Technology
Department of Computer Science & Engineering

Course: CSE-4508: RDBMS Programming Lab

Project Report

Vortex Gym Management System

Submitted To:

Sabrina Islam

Lecturer

Department of CSE

Submitted By:

Abdullah Al Fahad Diganta

ID: 220041210

Saom Bin Khaled

ID: 220041224

Md. Sazidur Rahman

ID: 220041226

Contents

1	Problem, Scope, and Project Overview	2
1.1	Problem Statement	2
1.2	Project Overview	2
1.3	Scope	2
2	List of Features	3
3	Tools and Technologies Used	4
4	Database Design	5
4.1	Core Design Principles and Normalization	5
4.2	Key Entities and Relationships	5
4.3	Diagram and Visuals	6
5	Advanced Queries, Procedural SQL, and Concepts	7
5.1	Database Triggers	7
5.2	Stored PL/pgSQL Functions	10
5.3	Advanced Analytical SQL Queries	13
5.4	Advanced Concepts Summary	16
6	Limitations of the Project	17
7	Modifications to Initial Scope & Design	17
8	Individual Contribution	18
9	Github Link	18

1 Problem, Scope, and Project Overview

1.1 Problem Statement

Managing modern gym facilities involves handling a multitude of operations ranging from member registration, subscription billing, and class scheduling to tracking trainer availability and maintaining equipment. Many existing solutions are either fragmented across different software systems or lack the sophisticated automation needed to handle real-time validations, such as class capacity limits, overlap checking, and payment synchronization. This fragmentation leads to operational inefficiencies, billing discrepancies, double-booking classes, and a suboptimal member experience.

1.2 Project Overview

The **Vortex Gym Management System** is an enterprise-grade full-stack web application designed to centralize and automate gym operations. It provides a unified platform supporting four distinct user roles: Administrative, Staff, Trainer, and Member. The system seamlessly handles end-to-end gym operations including secure authentication, membership purchases, dynamic class scheduling, real-time facility access via QR codes, personnel management, and equipment tracking.

1.3 Scope

The scope of the project encompasses:

- A web-based portal mapping out dashboards for Members, Trainers, Staff, and Admins.
- Integration of a real payment gateway (SSLCommerz) to handle online subscription transactions.
- Database-level enforcement of business rules, such as trainer shift validations and room capacity constraints using triggers and advanced queries.
- Analytics dashboards for business tracking, including Monthly Recurring Revenue (MRR) and usage patterns.
- A robust backend API using Spring Boot paired with a responsive React frontend.

2 List of Features

- **Authentication & Access Control:** Implements a secure JSON Web Token (JWT) based authentication system with robust role-based access control (RBAC). Upon login, users are routed to their respective role-specific dashboards (Admin, Staff, Trainer, or Member). It also includes a secure, token-based password reset mechanism sent via email, ensuring account recovery is safe and user-friendly.
- **Member Portal & Store:** A comprehensive self-service dashboard designed for gym members. Members can seamlessly view active subscriptions, browse and purchase membership plans or class packages (monthly/yearly), and interact with a dynamic calendar to book classes. It also features profile management (including profile photo uploads) and generates a unique, dynamic QR code for automated facility check-ins.
- **Admin Control Panel:** The central command hub for gym management. It features detailed revenue analytics (such as Monthly Recurring Revenue and peak-time usage charts), active/inactive membership tracking, and comprehensive inventory status monitoring. Admins also oversee staff operations, review trainer applications, and monitor the Live Facility Access feed to track real-time member check-ins.
- **Master Schedule Builder:** A powerful scheduling engine that enables administrators to define recurring class plans (e.g., weekly Yoga or HIIT sessions). The system automatically validates against strict constraints, preventing room double-booking and ensuring trainers are not double-booked or assigned outside their working shifts.
- **Trainer Dashboard:** A personalized portal for fitness instructors. Trainers can view their upcoming shift schedules, check details for their assigned classes, and easily perform digital roll-calls by marking attendance for enrolled class members. The dashboard also supports waitlist visibility, helping trainers manage class capacities effectively.
- **Staff Portal:** Designed for front-desk and maintenance personnel, this portal provides a live view of the check-in roster, allowing staff to monitor facility entry. Additionally, it equips staff with tools to manage physical equipment inventory, such as flagging malfunctioning machines for maintenance and logging repair statuses.
- **Recruitment System:** A fully integrated hiring workflow featuring a public-facing careers page where prospective trainers can submit their applications and initial details. This is paired with an administrative review board that allows management to instantly approve, reject, or promote applicants directly to the Trainer role, streamlining the onboarding process.
- **Automated Subscriptions & Grace Periods:** Seamless lifecycle management for memberships. The system automatically tracks subscription expiry dates, plan limits, and grace periods. This module is tightly synchronized with the SSLCommerz payment gateway via webhooks, ensuring that plans are instantly activated or updated the moment a payment succeeds, without manual intervention.
- **In-App Notifications & Alerts:** A real-time notification engine that pushes system alerts directly to member dashboards based on interactions (e.g., booking confirmations, waitlist promotions). It also utilizes Spring Mail to send automated SMTP email updates for critical events like payment receipts and application status changes.

3 Tools and Technologies Used

Frontend:

- **Framework:** React (Vite environment)
- **Routing & API:** React Router, Axios
- **Styling:** Tailwind CSS (Responsive mobile-first mapping with Dark Mode support)

Backend:

- **Framework:** Spring Boot 3 (Java 21)
- **Security:** Spring Security with JWT tokens
- **Data Access:** Spring Data JPA / Hibernate
- **Integrations:** SSLCommerz Payment API, Spring Mail (SMTP)
- **Tools:** Maven, Spring DevTools, Lombok, MapStruct

Database:

- **Engine:** PostgreSQL
- **Migration:** Flyway

4 Database Design

The relational schema is meticulously designed in PostgreSQL to support a robust, enterprise-grade gym management system, comprising 17 tables that strictly adhere to normalization principles (up to the Third Normal Form, 3NF). It effectively segregates user data, scheduling operations, financial transactions, and facility assets.

4.1 Core Design Principles and Normalization

- **Data Integrity & Consistency:** Foreign keys enforce strict referential integrity across all operational tables. Automated cascaded deletions (`ON DELETE CASCADE`) are cautiously applied to child records like `class_bookings` and `trainer_commissions` when a parent session or user is removed.
- **Security & Auditing:** Security-critical data, including `password_hash` and granular privilege roles, are stored in the core `users` table. Audit trail timestamps (`created_at`, `updated_at`) are present across major transactional tables to track historical changes.
- **Strategic Denormalization for Query Performance:** While highly normalized, specific columns like `current_status` in the `users` table are intentionally denormalized. This is seamlessly maintained by a PostgreSQL trigger (`sync_member_status`) summarizing the `subscriptions` table, significantly speeding up heavily-accessed dashboard queries by eliminating repetitive nested joins.

4.2 Key Entities and Relationships

- **Users & Authentication (`users`, `password_reset_tokens`):** The `users` table acts as the foundational entity, uniquely storing Members, Trainers, Admins, and Staff distinguished by a `role` enum. It links to `password_reset_tokens` to handle secure account recoveries in a one-to-many relationship.
- **Sales & Subscriptions (`membership_plans`, `subscriptions`, `invoices_payments`):** `membership_plans` acts as the master catalog holding pricing and recurring schedule formulas. A member's `subscriptions` represent an active lifecycle mapped directly to these plans. The `invoices_payments` table serves as the definitive financial ledger, recording transaction events synchronized with the SSLCommerz gateway.
- **Class Operations & Booking Engine (`class_sessions`, `class_bookings`, `rooms`):** The operational core where a `class_session` unifies a `room`, a `trainer` (from `users`), and a discrete time-slot. Members generate `class_bookings` (resolving a many-to-many relationship between users and sessions), which dynamically track ENROLLED or WAITLISTED statuses.
- **Workforce Management (`trainer_shifts`, `trainer_commissions`, `staff_payments`, `check_ins`):** `trainer_shifts` define valid working hours for instructors mapped to weekdays. The system calculates payroll dynamically by logging physical facility presence into `check_ins`, which subsequently feeds into the calculation of `staff_payments` and class-based `trainer_commissions`.

- **Physical Assets & Feedback (equipment, trainer_reviews, notifications):** equipment is structured as a standalone inventory tracking table for maintenance logs. trainer_reviews allow members to rate their instructors after classes, linking directly back to the trainer's user ID. notifications function as a scalable messaging queue for pushing real-time alerts to specific users.
- **Recruitment Pipeline (trainer_applications):** A purposefully decoupled table designed to hold public form data without creating clutter in the core users table until an admin officially approves and migrates an applicant via the dashboard.

4.3 Diagram and Visuals

The following Entity Relationship Diagram illustrates the interconnectivity among the 17 tables, highlighting the relationships and structure shaping the overall schema.

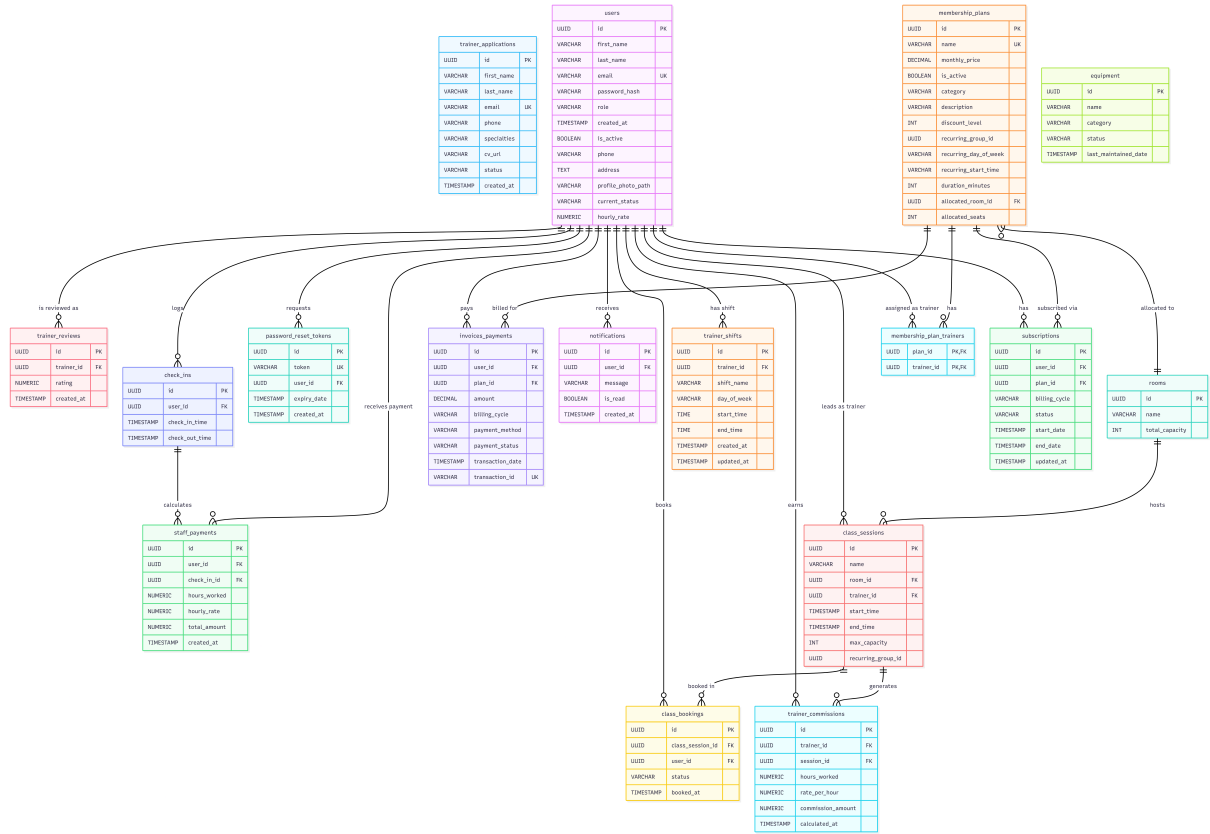


Figure 1: Entity Relationship Diagram of the Vortex Gym Management System

5 Advanced Queries, Procedural SQL, and Concepts

5.1 Database Triggers

We heavily implemented PostgreSQL triggers to enforce strict business rules directly at the database level, completely decoupled from the application service layer. This ensures data integrity even for raw SQL operations bypassing the backend.

Trigger 1: activate_subscription_after_payment

Table: invoices_payments **Event:** AFTER INSERT OR UPDATE

When a payment record's `payment_status` transitions to 'SUCCESS', this trigger function automatically provisions or renews a row in the `subscriptions` table. It computes the correct `start_date` and `end_date` based on whether the billing cycle is `MONTHLY` or `YEARLY`. If an existing subscription already exists for that user-plan pair, it is updated atomically; otherwise a new one is inserted. This eliminates the need for the backend to manually handle post-payment subscription activation.

```
1 CREATE OR REPLACE FUNCTION trg_process_successful_payment()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     new_end_date TIMESTAMP WITH TIME ZONE;
5     existing_sub_id UUID;
6 BEGIN
7     IF NEW.payment_status = 'SUCCESS' AND
8       (TG_OP = 'INSERT' OR OLD.payment_status <> 'SUCCESS') THEN
9         IF NEW.billing_cycle = 'YEARLY' THEN
10             new_end_date := CURRENT_TIMESTAMP + INTERVAL '1 year';
11         ELSE
12             new_end_date := CURRENT_TIMESTAMP + INTERVAL '1 month';
13         END IF;
14         SELECT id INTO existing_sub_id FROM subscriptions
15         WHERE user_id = NEW.user_id AND plan_id = NEW.plan_id LIMIT 1;
16         IF existing_sub_id IS NOT NULL THEN
17             UPDATE subscriptions SET
18                 billing_cycle = NEW.billing_cycle, status = 'ACTIVE',
19                 start_date = CURRENT_TIMESTAMP, end_date = new_end_date
20             ,
21                 updated_at = CURRENT_TIMESTAMP
22             WHERE id = existing_sub_id;
23         ELSE
24             INSERT INTO subscriptions
25                 (user_id, plan_id, billing_cycle, status, start_date,
26                  end_date)
27                 VALUES (NEW.user_id, NEW.plan_id, NEW.billing_cycle,
28                         'ACTIVE', CURRENT_TIMESTAMP, new_end_date);
29         END IF;
30     END IF;
31     RETURN NEW;
32 END;
33 $$ LANGUAGE plpgsql;
34
35 CREATE TRIGGER activate_subscription_after_payment
36 AFTER INSERT OR UPDATE ON invoices_payments
```



```
35 FOR EACH ROW EXECUTE FUNCTION trg_process_successful_payment();
```

Listing 1: Trigger: activate_subscription_after_payment

Trigger 2: trg_enforce_class_capacity

Table: class_bookings **Event:** BEFORE INSERT OR UPDATE

Before a booking row is committed, this trigger queries the current count of 'ENROLLED' bookings for the same session. If the count equals or exceeds the session's max_capacity, the incoming booking status is silently overwritten from 'ENROLLED' to 'WAITLISTED', preventing overbooking entirely at the data layer.

```
1 CREATE OR REPLACE FUNCTION check_class_capacity()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     current_enrolled INT;
5     class_max INT;
6 BEGIN
7     IF NEW.status = 'ENROLLED' THEN
8         SELECT COUNT(*) INTO current_enrolled FROM class_bookings
9         WHERE class_session_id = NEW.class_session_id AND status = '
10        ENROLLED';
11        SELECT max_capacity INTO class_max
12        FROM class_sessions WHERE id = NEW.class_session_id;
13        IF current_enrolled >= class_max THEN
14            NEW.status := 'WAITLISTED';
15        END IF;
16    END IF;
17    RETURN NEW;
18 $$ LANGUAGE plpgsql;
19
20 CREATE TRIGGER trg_enforce_class_capacity
21 BEFORE INSERT OR UPDATE ON class_bookings
22 FOR EACH ROW EXECUTE FUNCTION check_class_capacity();
```

Listing 2: Trigger: trg_enforce_class_capacity

Trigger 3: validate_trainer_shift

Table: class_sessions **Event:** BEFORE INSERT

Prevents scheduling a class outside the assigned trainer's registered working hours. It extracts the weekday and time from the session's start_time, then queries the trainer_shifts table for a valid overlapping shift. If no valid shift is found, a hard RAISE EXCEPTION blocks the insertion, ensuring all class sessions are always within sanctioned trainer availability.

```
1 CREATE OR REPLACE FUNCTION validate_trainer_shift()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     shift_exists BOOLEAN;
5     day_name VARCHAR(20);
6     class_start_time TIME;
7     class_end_time TIME;
8 BEGIN
9     day_name := CASE TO_CHAR(NEW.start_time, 'D')::INT
```

```

10         WHEN 1 THEN 'SUNDAY'    WHEN 2 THEN 'MONDAY'
11         WHEN 3 THEN 'TUESDAY'  WHEN 4 THEN 'WEDNESDAY'
12         WHEN 5 THEN 'THURSDAY' WHEN 6 THEN 'FRIDAY'
13         WHEN 7 THEN 'SATURDAY'
14     END;
15     class_start_time := NEW.start_time::TIME;
16     class_end_time   := NEW.end_time::TIME;
17     SELECT EXISTS(
18         SELECT 1 FROM trainer_shifts
19         WHERE  trainer_id = NEW.trainer_id
20             AND day_of_week = day_name
21             AND start_time <= class_start_time
22             AND end_time   >= class_end_time
23     ) INTO shift_exists;
24     IF NOT shift_exists THEN
25         RAISE EXCEPTION
26             'Class scheduled outside trainer shift. Trainer: %, Day: %'
27             ,
28             NEW.trainer_id, day_name;
29     END IF;
30     RETURN NEW;
31 END;
32 $$ LANGUAGE plpgsql;
33 CREATE TRIGGER validate_trainer_shift
34 BEFORE INSERT ON class_sessions
35 FOR EACH ROW EXECUTE FUNCTION validate_trainer_shift();

```

Listing 3: Trigger: validate_trainer_shift

Trigger 4: sync_member_status

Table: subscriptions **Event:** AFTER INSERT OR UPDATE

This trigger implements **strategic denormalization**. Whenever a subscription's status changes, it immediately reflects the new status into the `current_status` column on the `users` table. This avoids a repeated expensive JOIN to `subscriptions` on every member dashboard API call, providing a real-time, pre-aggregated status flag at the user record level.

```

1 CREATE OR REPLACE FUNCTION sync_member_status()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     member_status VARCHAR(20);
5 BEGIN
6     SELECT status INTO member_status FROM subscriptions
7     WHERE user_id = NEW.user_id ORDER BY updated_at DESC LIMIT 1;
8     member_status := COALESCE(member_status, 'INACTIVE');
9     UPDATE users SET current_status = member_status WHERE id = NEW.
10    user_id;
11    RETURN NEW;
12 END;
13 $$ LANGUAGE plpgsql;
14 CREATE TRIGGER sync_member_status
15 AFTER INSERT OR UPDATE ON subscriptions
16 FOR EACH ROW EXECUTE FUNCTION sync_member_status();

```

Listing 4: Trigger: sync_member_status

5.2 Stored PL/pgSQL Functions

Function 1: book_class_slot

Returns: JSON

An atomic, multi-validation booking procedure. It sequentially executes: (1) a **duplicate booking guard**, (2) a **time-overlap conflict check** against all other sessions the member is enrolled in, and (3) a **capacity pre-check** to determine ENROLLED vs WAITLISTED status before writing to class_bookings. The result is a structured JSON response, enabling clean API integration.

```
1 CREATE OR REPLACE FUNCTION book_class_slot(p_user_id UUID, p_session_id
  UUID)
2 RETURNS JSON AS $$
3 DECLARE
4     v_conflict_count    INT;    v_current_enrolled INT;    v_class_max INT
5     ;
6     v_booking_status    VARCHAR(20);    v_new_booking_id UUID;
7     v_session_start     TIMESTAMP WITH TIME ZONE;
8     v_session_end       TIMESTAMP WITH TIME ZONE;
9 BEGIN
10    -- 1. Duplicate Guard
11    IF EXISTS (SELECT 1 FROM class_bookings
12              WHERE class_session_id = p_session_id AND user_id =
13                p_user_id)
14    THEN RETURN json_build_object('success',false,'error','Already
15      booked.');
```

```
16    END IF;
17    -- 2. Load session window
18    SELECT start_time, end_time INTO v_session_start, v_session_end
19    FROM class_sessions WHERE id = p_session_id;
20    -- 3. Schedule Overlap Check
21    SELECT COUNT(*) INTO v_conflict_count
22    FROM class_bookings cb JOIN class_sessions cs ON cb.
23      class_session_id = cs.id
24      WHERE cb.user_id = p_user_id
25      AND cb.status IN ('ENROLLED','WAITLISTED')
26      AND cs.id != p_session_id
27      AND cs.start_time < v_session_end
28      AND cs.end_time > v_session_start;
29    IF v_conflict_count > 0 THEN
30      RETURN json_build_object('success',false,'error','Time conflict
31        .');
```

```
32    END IF;
33    -- 4. Capacity Resolution
34    SELECT COUNT(*) INTO v_current_enrolled FROM class_bookings
35    WHERE class_session_id = p_session_id AND status = 'ENROLLED';
36    SELECT max_capacity INTO v_class_max FROM class_sessions WHERE id=
37      p_session_id;
38    v_booking_status := CASE WHEN v_current_enrolled >= v_class_max
39      THEN 'WAITLISTED' ELSE 'ENROLLED' END;
40    -- 5. Atomic Insert (capacity trigger re-validates independently)
41    INSERT INTO class_bookings (class_session_id, user_id, status)
42    VALUES (p_session_id, p_user_id, v_booking_status)
43    RETURNING id, status INTO v_new_booking_id, v_booking_status;
44    RETURN json_build_object('success',true,'bookingId',
45      v_new_booking_id,
46      'status',v_booking_status);
```

```

40 EXCEPTION WHEN OTHERS THEN
41     RETURN json_build_object('success',false,'error',SQLERRM);
42 END;
43 $$ LANGUAGE plpgsql;

```

Listing 5: Function: book_class_slot

Function 2: assign_trainer (with UPSERT)

Returns: JSON

Reassigns a trainer to a session and atomically upserts a commission record. Uses EXTRACT EPOCH to convert session duration to hours, computes commission, and then employs ON CONFLICT (session_id) DO UPDATE to guarantee a single commission row per session — a safer atomic merge versus a separate SELECT-then-INSERT.

```

1 CREATE OR REPLACE FUNCTION assign_trainer(
2     p_session_id UUID, p_trainer_id UUID, p_rate_per_hour NUMERIC
3     DEFAULT 15.00)
4 RETURNS JSON AS $$
5 DECLARE
6     v_hours_worked      NUMERIC;
7     v_commission_amount NUMERIC;
8 BEGIN
9     IF NOT EXISTS (SELECT 1 FROM users WHERE id = p_trainer_id
10                    AND role = 'TRAINER' AND is_active = TRUE) THEN
11         RETURN json_build_object('success',false,'error','Trainer not
12 found. ');
13     END IF;
14     SELECT EXTRACT(EPOCH FROM (end_time - start_time)) / 3600.0
15     INTO v_hours_worked FROM class_sessions WHERE id = p_session_id;
16     v_commission_amount := ROUND(v_hours_worked * p_rate_per_hour, 2);
17     UPDATE class_sessions SET trainer_id = p_trainer_id WHERE id =
18     p_session_id;
19     -- UPSERT: one commission record per session (UNIQUE on session_id)
20     INSERT INTO trainer_commissions
21     (trainer_id, session_id, hours_worked, rate_per_hour,
22     commission_amount)
23     VALUES (p_trainer_id, p_session_id, v_hours_worked,
24     p_rate_per_hour, v_commission_amount)
25     ON CONFLICT (session_id) DO UPDATE SET
26     trainer_id          = EXCLUDED.trainer_id,
27     hours_worked        = EXCLUDED.hours_worked,
28     commission_amount   = EXCLUDED.commission_amount,
29     calculated_at       = CURRENT_TIMESTAMP;
30     RETURN json_build_object('success',true,'commissionAmount',
31     v_commission_amount);
32 EXCEPTION WHEN OTHERS THEN
33     RETURN json_build_object('success',false,'error',SQLERRM);
34 END;
35 $$ LANGUAGE plpgsql;

```

Listing 6: Function: assign_trainer with ON CONFLICT UPSERT

Function 3: get_used_capacity_for_room_at_time

Returns: INTEGER

Sums the max_capacity of all class sessions that overlap with a proposed time window

in a given room. Used during schedule creation to prevent exceeding a room's physical spatial limits with multiple overlapping sessions.

```
1 CREATE OR REPLACE FUNCTION get_used_capacity_for_room_at_time(  
2     p_room_id UUID, p_start_time TIMESTAMPTZ, p_end_time TIMESTAMPTZ)  
3 RETURNS INTEGER AS $$  
4 DECLARE v_used_capacity INTEGER;  
5 BEGIN  
6     SELECT COALESCE(SUM(cs.max_capacity), 0) INTO v_used_capacity  
7     FROM class_sessions cs  
8     WHERE cs.room_id = p_room_id  
9           AND cs.start_time < p_end_time  
10          AND cs.end_time > p_start_time;  
11     RETURN v_used_capacity;  
12 END;  
13 $$ LANGUAGE plpgsql STABLE;
```

Listing 7: Function: get_used_capacity_for_room_at_time

Function 4: count_overlapping_trainer_classes

Returns: BIGINT

Counts how many sessions a trainer already has that overlap with a new proposed time slot. An optional `p_exclude_session_id` parameter avoids self-conflict when editing an existing session. Called as a pre-validation guard before every scheduling write to enforce trainer exclusivity.

```
1 CREATE OR REPLACE FUNCTION count_overlapping_trainer_classes(  
2     p_trainer_id UUID, p_start_time TIMESTAMPTZ, p_end_time TIMESTAMPTZ  
3     ,  
4     p_exclude_session_id UUID DEFAULT NULL)  
5 RETURNS BIGINT AS $$  
6 DECLARE v_overlap_count BIGINT;  
7 BEGIN  
8     SELECT COUNT(*) INTO v_overlap_count FROM class_sessions cs  
9     WHERE cs.trainer_id = p_trainer_id  
10          AND cs.start_time < p_end_time  
11          AND cs.end_time > p_start_time  
12          AND (p_exclude_session_id IS NULL OR cs.id <>  
13              p_exclude_session_id);  
14     RETURN v_overlap_count;  
15 END;  
16 $$ LANGUAGE plpgsql STABLE;
```

Listing 8: Function: count_overlapping_trainer_classes

Function 5: get_member_available_classes

Returns: TABLE

Returns all future class sessions that fall within a member's active subscription date window, joining room and trainer data for a complete calendar payload. A subscription-gate is enforced at the database level: members without an **ACTIVE** subscription see zero rows, requiring no application-side filtering.

```
1 CREATE OR REPLACE FUNCTION get_member_available_classes(p_user_id UUID)  
2 RETURNS TABLE (session_id UUID, name VARCHAR, start_time TIMESTAMPTZ,  
3                 end_time TIMESTAMPTZ, max_capacity INT, room_id UUID,
```

```

4         room_name VARCHAR, trainer_id UUID, trainer_first_name
    VARCHAR,
5         trainer_last_name VARCHAR, remaining_capacity BIGINT) AS
    $$
6 BEGIN
7     RETURN QUERY
8     SELECT cs.id, cs.name, cs.start_time, cs.end_time, cs.max_capacity,
9            rm.id, rm.name, tr.id, tr.first_name, tr.last_name,
10           cs.max_capacity::BIGINT
11     FROM class_sessions cs
12     JOIN rooms rm ON rm.id = cs.room_id
13     JOIN users tr ON tr.id = cs.trainer_id
14     WHERE cs.start_time > CURRENT_TIMESTAMP
15           AND EXISTS (
16             SELECT 1 FROM subscriptions s
17             WHERE s.user_id = p_user_id AND s.status = 'ACTIVE'
18                   AND cs.start_time BETWEEN
19                   COALESCE(s.start_date, CURRENT_TIMESTAMP - INTERVAL '1
20 day')
21                   AND COALESCE(s.end_date, CURRENT_TIMESTAMP + INTERVAL '
22 5 years'))
23     ORDER BY cs.start_time ASC;
24 END;
25 $$ LANGUAGE plpgsql STABLE;

```

Listing 9: Function: get_member_available_classes

5.3 Advanced Analytical SQL Queries

Query 1: peak_hours_analysis (GROUPING SETS)

Returns: TABLE

Aggregates check-in logs using **GROUPING SETS** — a powerful SQL extension that computes multiple aggregation levels within a single query scan. It simultaneously produces counts per day-hour combination, per-day totals, per-hour totals (for the peak-hour bar chart), and an overall grand total. This avoids four separate UNION ALL queries.

```

1 CREATE OR REPLACE FUNCTION peak_hours_analysis()
2 RETURNS TABLE (day_label TEXT, hour_label INTEGER,
3                checkins_count BIGINT, grouping_level TEXT) AS $$
4 BEGIN
5     RETURN QUERY
6     SELECT
7         CASE WHEN GROUPING(day_name)=1 THEN 'ALL_DAYS' ELSE day_name
8     END,
9         CASE WHEN GROUPING(hour_val) =1 THEN NULL           ELSE hour_val
10    END,
11    COUNT(*)::BIGINT,
12    CASE
13        WHEN GROUPING(day_name)=0 AND GROUPING(hour_val)=0 THEN '
14 DAY_HOUR '
15        WHEN GROUPING(day_name)=0 AND GROUPING(hour_val)=1 THEN '
16 DAY_TOTAL '
17        WHEN GROUPING(day_name)=1 AND GROUPING(hour_val)=0 THEN '
18 HOUR_TOTAL '
19        ELSE 'GRAND_TOTAL '
20    END

```

```

16 FROM (SELECT TO_CHAR(ci.check_in_time, 'Dy') AS day_name,
17             EXTRACT(HOUR FROM ci.check_in_time)::INT AS hour_val
18         FROM check_ins ci) attendance
19 GROUP BY GROUPING SETS ((day_name, hour_val), (day_name), (hour_val
20 ), ())
21 ORDER BY CASE WHEN GROUPING(day_name)=1 THEN 1 ELSE 0 END,
22            day_label, hour_label;
23 END;
24 $$ LANGUAGE plpgsql STABLE;

```

Listing 10: Function: peak_hours_analysis using GROUPING SETS

Query 2: top_trainers_ranking (CTEs + DENSE_RANK Window Function)

Returns: TABLE

Uses a chain of **Common Table Expressions (CTEs)** to decompose a multi-step aggregation plan, followed by the **DENSE_RANK** window function to generate a trainer leaderboard based on class attendance retention rate. Tied trainers share the same rank position, which is semantically correct for a competitive ranking (unlike ROW_NUMBER).

```

1 CREATE OR REPLACE FUNCTION top_trainers_ranking()
2 RETURNS TABLE (trainer_id UUID, first_name VARCHAR, last_name VARCHAR,
3                 attended_count BIGINT, attendance_events BIGINT,
4                 retention_rate NUMERIC, rank_position BIGINT) AS $$
5 BEGIN
6     RETURN QUERY
7     WITH trainer_attendance AS (
8         SELECT cs.trainer_id,
9                SUM(CASE WHEN cb.status = 'PRESENT' THEN 1 ELSE 0 END)::
10                BIGINT AS attended_count,
11                SUM(CASE WHEN cb.status IN ('PRESENT', 'ABSENT') THEN 1 ELSE
12                0 END)::BIGINT AS attendance_events
13                FROM class_sessions cs
14                LEFT JOIN class_bookings cb ON cb.class_session_id = cs.id
15                GROUP BY cs.trainer_id
16            ),
17            trainer_scores AS (
18                SELECT ta.trainer_id, ta.attended_count, ta.attendance_events,
19                       COALESCE(ROUND((ta.attended_count * 100.0)
20                       / NULLIF(ta.attendance_events, 0), 2), 0)
21                AS retention_rate
22                FROM trainer_attendance ta
23            )
24            SELECT u.id, u.first_name, u.last_name,
25                   ts.attended_count, ts.attendance_events, ts.retention_rate,
26                   DENSE_RANK() OVER (ORDER BY ts.retention_rate DESC,
27                                           ts.attendance_events DESC)
28            FROM trainer_scores ts
29            JOIN users u ON u.id = ts.trainer_id
30            WHERE u.role = 'TRAINER'
31            ORDER BY rank_position, u.first_name, u.last_name;
32 END;
33 $$ LANGUAGE plpgsql STABLE;

```

Listing 11: Function: top_trainers_ranking with CTEs and DENSE_RANK

Query 3: monthly_revenue_rollup (ROLLUP)

Returns: TABLE

Uses the **ROLLUP** operator to generate a hierarchical revenue summary — per-month-per-plan-category rows and a per-month subtotal — in a single query pass. This powers the revenue breakdown panel on the Admin Dashboard, distinguishing between **BASE_MEMBERSHIP** and class-tier revenues automatically.

```
1 CREATE OR REPLACE FUNCTION monthly_revenue_rollup()
2 RETURNS TABLE (revenue_month DATE, plan_type TEXT, total_income NUMERIC
3 ) AS $$
4 BEGIN
5     RETURN QUERY
6     SELECT
7         DATE_TRUNC('month', ip.transaction_date)::DATE AS revenue_month
8         ,
9         COALESCE(mp.category, 'ALL_PLAN_TYPES')::TEXT AS plan_type,
10        COALESCE(SUM(ip.amount), 0) AS total_income
11 FROM invoices_payments ip
12 JOIN membership_plans mp ON mp.id = ip.plan_id
13 WHERE ip.payment_status = 'SUCCESS'
14 GROUP BY DATE_TRUNC('month', ip.transaction_date), ROLLUP(mp.
15 category)
16 ORDER BY revenue_month DESC, plan_type;
17 END;
18 $$ LANGUAGE plpgsql STABLE;
```

Listing 12: Function: monthly_revenue.rollup using ROLLUP

Query 4: debtor_list_active (Correlated CTEs)

Returns: TABLE

Surfaces members who are physically inside the facility (open check-in: **check_out_time** IS NULL) while also having overdue invoices. Uses two independent CTEs — one for active check-ins and one for overdue payments — joined against the **users** table. A practical operational query for front-desk staff alerting.

```
1 CREATE OR REPLACE FUNCTION debtor_list_active()
2 RETURNS TABLE (user_id UUID, first_name VARCHAR, last_name VARCHAR,
3 email VARCHAR,
4 latest_check_in TIMESTAMPTZ, overdue_invoices BIGINT,
5 overdue_amount NUMERIC) AS $$
6 BEGIN
7     RETURN QUERY
8     WITH active_checkins AS (
9         SELECT ci.user_id, MAX(ci.check_in_time) AS latest_check_in
10        FROM check_ins ci WHERE ci.check_out_time IS NULL
11        GROUP BY ci.user_id
12    ),
13    overdue AS (
14        SELECT ip.user_id,
15               COUNT(*)::BIGINT AS overdue_invoices,
16               COALESCE(SUM(ip.amount), 0) AS overdue_amount
17        FROM invoices_payments ip WHERE ip.payment_status = 'OVERDUE'
18        GROUP BY ip.user_id
19    )
20     SELECT u.id, u.first_name, u.last_name, u.email,
```



```

20         ac.latest_check_in, od.overdue_invoices, od.overdue_amount
21 FROM active_checkins ac
22 JOIN overdue od ON od.user_id = ac.user_id
23 JOIN users u ON u.id = ac.user_id
24 ORDER BY ac.latest_check_in DESC;
25 END;
26 $$ LANGUAGE plpgsql STABLE;

```

Listing 13: Function: `debtor_list_active` using Correlated CTEs

5.4 Advanced Concepts Summary

- **Strategic Denormalization + Trigger Sync:** The `users.current_status` column caches each member’s latest subscription state, maintained automatically by the `sync_member_status` trigger. This eliminates the need to JOIN `subscriptions` on every member dashboard request, trading a minimal write overhead for significant read-path performance.
- **UPSERT with ON CONFLICT:** The `assign_trainer` function uses `INSERT ... ON CONFLICT (session_id) DO UPDATE` to atomically merge commission records. This eliminates race conditions present in a naive `SELECT-then-INSERT` pattern and ensures exactly one commission row per session.
- **GROUPING SETS & ROLLUP:** Used in `peak_hours_analysis()` and `monthly_revenue_rollup` respectively. These constructs compute multiple GROUP BY hierarchies in one query pass, avoiding the performance cost and redundancy of chained UNION ALL statements.
- **Window Functions (DENSE_RANK):** Applied in `top_trainers_ranking()` to generate a shared-rank competitive leaderboard based on trainer retention rate. Produces semantically correct results for ties, unlike `ROW_NUMBER`.
- **Common Table Expressions (CTEs):** Used across multiple analytical functions to decompose complex multi-step aggregations into named, readable WITH blocks. CTEs improve maintainability, are often materialized by the planner for intermediate result reuse, and keep function bodies clean.
- **Composite Indexing for Query Optimization:** Targeted multi-column indexes were created to match the query planner’s access patterns. Key examples: `idx_class_sessions_room` (`room_id`, `start_time`, `end_time`) for room overlap checks, and `idx_subscriptions_user_status` for member calendar lookups, dramatically reducing full-table scans.
- **Schema Versioning via Flyway:** All DDL changes, trigger/function definitions, and seed data are managed through sequentially numbered Flyway migration files (V1 through V13), providing a deterministic, auditable, and reproducible database history critical for team collaboration and CI/CD deployments.

6 Limitations of the Project

The following limitations were identified as deliberate scoping decisions made to keep the project focused on core RDBMS concepts and deliverable within the semester timeline.

- **Single-Location Architecture:** The database schema is intentionally designed around a single physical gym facility. Supporting a multi-branch franchise model would require introducing a `facilities` entity and refactoring nearly all foreign key relationships to be facility-scoped — a significant schema redesign that was outside the project’s intended scope.
- **No Table Partitioning:** High-volume tables such as `check_ins` and `invoices_payments` are not partitioned by date range. As data grows over months or years, query performance on these tables would degrade. PostgreSQL’s native range partitioning (e.g., monthly partitions on `check_in_time`) would be the recommended production optimization.
- **Software-Based QR Check-In:** The QR code facility access system operates entirely as a software scan within the browser. A production deployment would require integration with physical IoT turnstile hardware and a dedicated embedded reader — a hardware layer outside the scope of this RDBMS-focused project.
- **Limited Predictive Analytics:** The dashboard provides descriptive statistics (MRR, peak hours, churn rate) computed via SQL aggregations. However, predictive capabilities such as churn forecasting or personalized class recommendations would require machine learning pipelines operating outside of PostgreSQL, which was beyond the project’s database-centric focus.

7 Modifications to Initial Scope & Design

Throughout the semester’s development path, several modifications were made over the original scope:

- **Migration of Business Logic to Database:** Initially proposed to handle room booking availability and trainer validation strictly from Spring Boot services. We migrated these rules explicitly into PostgreSQL triggers (`validate_trainer_shift`, `check_class_capacity`) ensuring data consistency even from raw SQL inserts.
- **Payment Systems Transition:** Originally intended to implement a mock check-out protocol. We evolved this to incorporate the **SSLCommerz** integration for true localized payment handling, pushing WebHook validations into the system.
- **Refined User Roles:** We split the “Management” role down into separate **Admin** and **Staff** roles. This ensures granular security so general front-desk staff can only monitor attendance/inventory, while Admins deal with recurring financial plans.

8 Individual Contribution

Team Member	Contributions
Abdullah Al Fahad Diganta	Developed the Admin Dashboard analytics interface — revenue charts (MRR, peak hours, churn rate), membership tracking panels, and the responsive Tailwind-based layout system across all dashboards. Built the Trainer Dashboard end-to-end (shift schedule view, class roster, attendance marking UI and API). Authored the advanced analytical queries: <code>peak_hours_analysis</code> (GROUPING SETS), <code>top_trainers_ranking</code> (CTEs + DENSE_RANK), <code>monthly_revenue_rollup</code> (ROLLUP), and <code>debtor_list_active</code> .
Saom Bin Khaled	Implemented the JWT-based authentication system (backend security + login/register UI), SSLCommerz payment gateway integration (webhook handling + store frontend), and the Master Schedule Builder (room/trainer conflict validation logic, scheduling API, and admin scheduling UI). Built the Member Portal end-to-end — dashboard UI, subscription store, class calendar with booking flow, QR code generation, and profile photo uploads. Authored the <code>get_member_available_classes</code> function, <code>activate_subscription_after_payment</code> trigger, and the <code>validate_trainer_shift</code> trigger.
Md. Sazidur Rahman	Developed the Admin Control Panel's operational modules — trainer application review board (UI + approval/rejection API), membership plan CRUD management, and room/equipment configuration. Built the Staff Portal (live check-in roster, equipment inventory and maintenance flagging). Authored the <code>book_class_slot</code> stored function, <code>trg_enforce_class_capacity</code> and <code>sync_member_status</code> triggers. Designed the Flyway migration structure and ER Diagram. Integrated the Spring Mail SMTP notification system and the public-facing careers page.

9 Github Link

The fully documented source code of Vortex Gym Management System can be accessed at:

<https://github.com/greenbinjack/vortex-fitness-system>